GCP Deployment

Every Google Cloud resource that runs PhysioGuidance in me-central2 (Dammam) — services, jobs, databases, buckets, IAM, secrets, networking — verified against production the day of this handover.

PREPARED FOR

PhysioGuidance Client Team

DATE

2026-08-17

GCP PROJECT

`nelc-elearning-platform-479319`

PHYSIOGUIDANCE

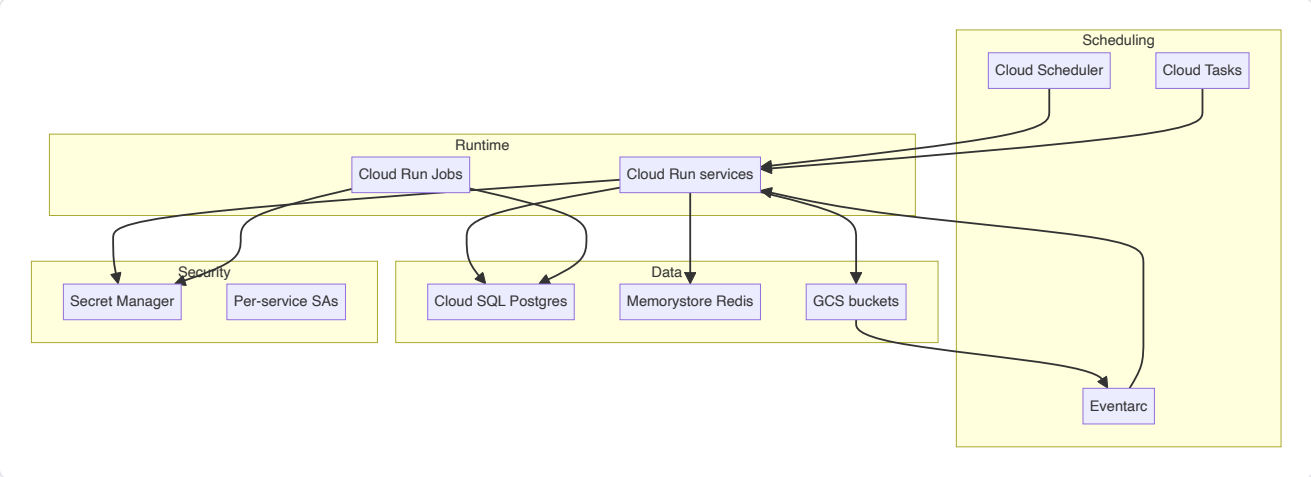
Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Bird's-eye view of the deployed GCP surface
2. Cloud Run services
3. Cloud Run Jobs
4. Cloud SQL — the one database
5. Memorystore Redis
6. GCS buckets
7. Cloud Tasks queues and Cloud Scheduler entries
8. Eventarc triggers
9. Per-service IAM (least-privilege service accounts)
10. Secret Manager
11. Networking (VPC connector, private IPs, edge TLS)
12. GitHub Actions to GCP (Workload Identity Federation)
13. Cost profile
14. Component status matrix
15. Rebuild-from-zero procedure
16. Where to look next

Deployed commit at the time of this handover: `d396b040`.

1. Bird's-eye view of the deployed GCP surface



Everything runs in `me-central2` (Dammam). Region enforcement lives in `infra/config.sh` — every deploy script sources that file and passes `--region=${GCP_REGION}` to every `gcloud` call.

---

2. Cloud Run services

Five services, each with its own deploy script under `infra/{service}/deploy.sh`.

Service	Deploy script	Runtime service account	Notes
`physioguidance-web`	`infra/frontend/deploy.sh`	`web-runtime@`	Next.js frontend, public
`physioguidance-api`	`infra/backend/deploy.sh`	`api-runtime@`	NestJS HTTP API
`physioguidance-api-worker`	`infra/worker/deploy.sh`	`worker-runtime@`	BullMQ consumers, internal-only (`--no-allow-unauthenticated`)
`physioguidance-cms`	`infra/strapi/deploy.sh`	`strapi-runtime@`	Strapi CMS
`physioguidance-malware-scanner`	`infra/malware-scanner/deploy.sh`	`malware-runtime@`	Eventarc-triggered ClamAV, internal-only

Per-service resource limits (from source)

Values come from `infra/config.sh` and the per-service deploy scripts.

Service	CPU	Memory	Min instances	Max instances	CPU throttling
`physioguidance-web`	1	512Mi	0	3	on (default)
`physioguidance-api`	1	1Gi	0	10	on (`--cpu-throttling`)
`physioguidance-api-worker`	1	1536Mi	1	1	off (`--no-cpu-throttling`)
`physioguidance-cms`	1	512Mi	0	2	on (default)
`physioguidance-malware-scanner`	2	4Gi	0	5	on (default)

Notes:

- The worker keeps `min=1/max=1` because BullMQ consumers must poll Redis continuously. CPU throttling is disabled on that service so ffmpeg subprocesses are not starved between HTTP requests.
- The api-worker container boots the full NestJS module tree; the 1.5 GiB memory ceiling is sized against a measured ~800 MiB resident-set floor (see comments in `infra/worker/deploy.sh`).
- The malware scanner runs at 4 GiB / 2 vCPU with a 540-second request timeout — enough headroom for ClamAV's signature database plus a scan of the 50 MiB upper file-size limit.
- Request concurrency and request timeout on the HTTP services use Cloud Run defaults (`gcloud` does not set `--concurrency` or `--timeout` in these scripts).

Traffic pinning

Every deploy script ends with `gcloud run services update-traffic --to-latest`. This defends against a previous rollback pinning traffic to a specific revision and silently keeping the new deploy dark.

---

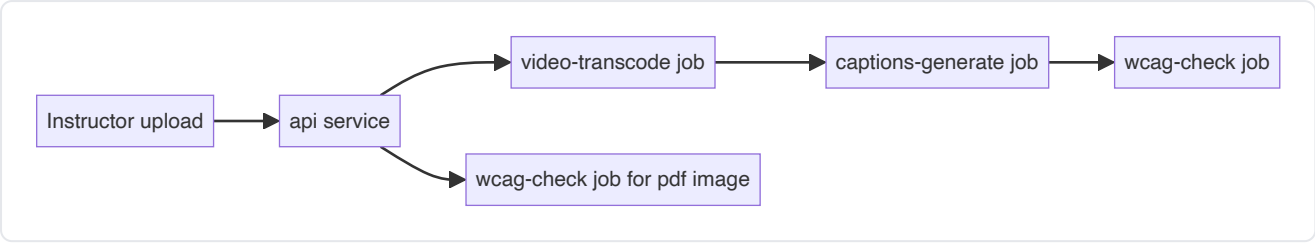
3. Cloud Run Jobs

Three Cloud Run Jobs run the heavy processing pipeline off the main HTTP service. All three run the same api-web image (`physioguidance-api:latest`) with a different `JOB_NAME` env var that resolves the correct handler inside `main-job.ts`.

Job	Provisioning script	CPU	Memory	Task timeout	Retries
`physio-job-video-transcode`	`infra/scripts/setup-video-transcode-job.sh`	2	2Gi	5400s (90 min)	2
`physio-job-captions-generate`	`infra/scripts/setup-captions-generate-job.sh`	1	1Gi	1800s (30 min)	2
`physio-job-wcag-check`	`infra/scripts/setup-wcag-check-job.sh`	1	512Mi	300s (5 min)	2

All three jobs attach the same Cloud SQL instance via `--set-cloudsql-instances` and route egress through the `physio-vpc-connector-me` connector. They are executed on-demand by the api-web service through `CloudRunJobsService.runJob(...)` — see `backends/src/modules/processing/processing.service.ts` for the dispatch call sites.

Trigger chain:



4. Cloud SQL — the one database

Verified from `infra/config.sh` and `infra/scripts/setup-database.sh`.

Property	Value	Source
Instance name	`physioguidance-db-me`	`infra/config.sh` `DB_INSTANCE_NAME`
Engine	PostgreSQL 15	`infra/config.sh` `DB_VERSION=POSTGRES_15`
Tier	`db-g1-small`	`infra/config.sh` `DB_TIER`
Region	`me-central2`	`infra/config.sh` `GCP_REGION`
Connection name	`nelc-elearning-platform-479319:me-central2:physioguidance-db-me`	Assembled by every deploy script
Database	`physioguidance`	`DATABASE_NAME=physioguidance` in every runtime env
App user	`apiuser`	`infra/scripts/setup-database.sh`
Strapi database	`strapi` (same instance, different DB)	`infra/strapi/deploy.sh`

Access pattern

- Cloud Run services and Jobs attach via `--add-cloudsql-instances=<connectionName>`, which mounts a Unix socket at `/cloudsql/<connectionName>/.s.PGSQL.5432`. `DATABASE_URL` (in Secret Manager) references that socket path.
- Local development connects via `cloud-sql-proxy` — `infra/scripts/migrate-production.sh` starts the proxy on `localhost:5433`, rewrites `DATABASE_URL` to point at it, and runs `npm run db:migrate`.
- Backup exports are provisioned by `infra/scripts/setup-backups.sh`, which creates the `physioguidance-backups` bucket with a 30-day lifecycle rule and grants the Cloud SQL instance service account `roles/storage.objectAdmin` on that bucket.

Live instance attributes (backup retention, PITR window, deletion protection, SSL mode, availability type) are runtime state and should be confirmed on the day of any operational review with `gcloud sql instances describe physioguidance-db-me`.

---

5. Memorystore Redis

Runtime configuration referenced from every service that touches Redis:

Setting	Value	Source
Endpoint	Read from Secret Manager	`REDIS_HOST` secret
Port	6378	`REDIS_PORT=6378` in `infra/backend/deploy.sh`, `infra/worker/deploy.sh` and the Cloud Run Job scripts
TLS	Enabled	`REDIS_TLS=true` in the same scripts
CA cert	Read from Secret Manager	`REDIS_CA_CERT` secret

Redis carries the BullMQ queues consumed by `physioguidance-api-worker` and the chain steps between Cloud Run Jobs.

---

6. GCS buckets

Six buckets used at runtime. Names are hard-coded in every deploy script and in `backends/src/modules/storage`.

Bucket	Purpose	Malware scanned	Notable lifecycle
`physioguidance-public`	Invoices, transcoded materials, avatars	Yes	none
`physioguidance-private`	Payment receipts, PDPL exports, general user uploads	Yes	10-year retention on `bank-transfer-receipts/` prefix (Saudi accounting law)
`physioguidance-trainers`	Trainer CVs, certificates	Yes	none
`physioguidance-cms`	Strapi media	No (admin-authored)	none
`physioguidance-backups`	Cloud SQL export dumps	No	30-day lifecycle delete
`physioguidance-quarantine`	Files flagged by the scanner	Terminal state	uniform IAM, public-access-prevention on create

Provisioning:

- `infra/scripts/setup-storage.sh` creates the three user-facing buckets and applies a permissive CORS policy.
- `infra/scripts/setup-backups.sh` creates the backups bucket with the 30-day lifecycle rule.
- `infra/malware-scanner/deploy.sh` creates the quarantine bucket with `--uniform-bucket-level-access` and `--public-access-prevention`.
- `infra/scripts/setup-bank-receipt-retention.sh` merges the 10-year `matchesPrefix: bank-transfer-receipts/` lifecycle rule onto `physioguidance-private`.

---

7. Cloud Tasks queues and Cloud Scheduler entries

Cloud Tasks queues

Eight domain-scoped queues (`infra/scripts/setup-cloud-tasks.sh`). All target the same dispatch endpoint `POST /internal/tasks/dispatch` on `physioguidance-api`. Cloud Tasks signs an OIDC token as `tasks-dispatcher@`, which holds `roles/run.invoker` on the api service; the dispatch endpoint routes the payload to the matching TaskHandler.

Queue	Rate (per sec)	Concurrent	Max attempts	Max retry duration
`session-events`	10	20	10	86400s
`ticket-events`	10	20	10	86400s
`payment-events`	5	10	15	604800s
`maintenance-events`	5	10	10	86400s
`points-events`	5	10	10	86400s
`backup-events`	2	5	10	86400s
`report-events`	2	5	10	86400s
`system-events`	5	10	10	86400s

Cloud Scheduler jobs

Three scheduled HTTP jobs (`infra/scripts/setup-reconciliation-cron.sh`). All invoke `physioguidance-api` over OIDC using the `reconciliation-cron@` service account and pass an `X-Cron-Token` header verified server-side against the `RECONCILIATION_CRON_TOKEN` secret.

Job	Schedule (Asia/Riyadh)	Endpoint
`reconciliation-daily-scan`	`0 2 * * *`	`POST /api/payments/reconciliation/cron/daily-scan`
`reconciliation-stale-reminders`	`0 10 * * *`	`POST /api/payments/reconciliation/cron/stale-reminders`
`bnpl-status-poll`	`*/5 * * * *`	`POST /api/payments/reconciliation/cron/bnpl-status-poll`

The BNPL poll is the webhook fallback for Tabby and Tamara — it progresses any payment that is still pending after five minutes.

---

8. Eventarc triggers

Three triggers, all wired by `infra/malware-scanner/deploy.sh` . Each fires on `google.cloud.storage.object.v1.finalized` and delivers to `physioguidance-malware-scanner` as `malware-runtime@` .

Trigger	Source bucket
`malware-scan-physioguidance-private`	`physioguidance-private`
`malware-scan-physioguidance-public`	`physioguidance-public`
`malware-scan-physioguidance-trainers`	`physioguidance-trainers`

The trigger's `--service-account` (the impersonation identity used by Eventarc when invoking the destination) is refreshed to `malware-runtime@` on every re-run of `deploy.sh`, so a change of SA rolls out cleanly. `malware-runtime@` holds `roles/run.invoker` on the scanner service so Eventarc's authenticated delivery is accepted.

---

9. Per-service IAM (least-privilege service accounts)

Every Cloud Run service and every Cloud Run Job runs under a dedicated runtime service account, never the default compute SA. Two additional SAs handle scheduler and task dispatch.

Service account	Attached to	Grants applied by scripts
`web-runtime@`	`physioguidance-web`	None beyond the runtime default (frontend needs no GCP-side data access)
`api-runtime@`	`physioguidance-api`	Cloud SQL client, Secret Manager access to the mounted secrets, storage access to the three platform buckets (via bucket bindings in the setup scripts)
`worker-runtime@`	`physioguidance-api-worker`	Same shape as `api-runtime@` — the worker executes the same code paths against DB, Redis and storage
`strapi-runtime@`	`physioguidance-cms`	Cloud SQL client, Secret Manager access to the Strapi secrets, storage on the `physioguidance-cms` bucket only
`malware-runtime@`	`physioguidance-malware-scanner` and Eventarc triggers	`roles/storage.objectAdmin` on the three watched buckets plus `physioguidance-quarantine`, `roles/eventarc.eventReceiver` at project level, `roles/run.invoker` on its own service
`tasks-dispatcher@`	Cloud Tasks queues (OIDC identity)	`roles/run.invoker` on `physioguidance-api`
`reconciliation-cron@`	Cloud Scheduler jobs (OIDC identity)	`roles/run.invoker` on `physioguidance-api`

Verify anytime with:

```
gcloud projects get-iam-policy nelc-elearning-platform-479319 \
  --flatten="bindings[].members" \
  --filter="bindings.members:api-runtime@nelc-elearning-platform-
        479319.iam.gserviceaccount.com" \
  --format="value(bindings.role)"
```

10. Secret Manager

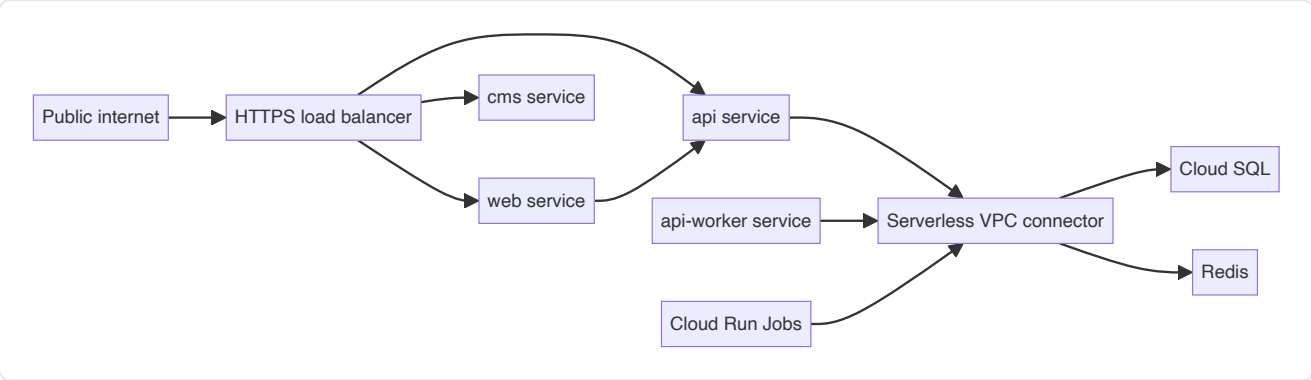
The runtime secrets are mounted onto each service via `--set-secrets` in the deploy scripts. They fall into four buckets:

Category	Secrets
Infrastructure credentials	`DATABASE_URL`, `DATABASE_PASSWORD`, `STRAPI_DATABASE_PASSWORD`, `REDIS_HOST`, `REDIS_CA_CERT`
Application-generated	`JWT_SECRET`, `JWT_REFRESH_SECRET`, `PII_ENCRYPTION_KEY`, `RECONCILIATION_CRON_TOKEN`, `STRAPI_APP_KEYS`, `STRAPI_API_TOKEN_SALT`, `STRAPI_ADMIN_JWT_SECRET`, `STRAPI_TRANSFER_TOKEN_SALT`, `STRAPI_JWT_SECRET`
Third-party APIs	Moyasar (test+live: `SECRET_KEY`, `PUBLISHABLE_KEY`, `WEBHOOK_SECRET`), Tabby (`TABBY_SECRET_KEY`, `TABBY_WEBHOOK_SECRET`), Tamara (`TAMARA_API_TOKEN`, `TAMARA_NOTIFICATION_TOKEN`, `TAMARA_PUBLIC_KEY`), Zoom (`ZOOM_ACCOUNT_ID`, `ZOOM_CLIENT_ID`, `ZOOM_CLIENT_SECRET`, `ZOOM_WEBHOOK_SECRET_TOKEN`), OpenAI (`OPENAI_API_KEY`), Resend (`RESEND_API_KEY`), SMTP (`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`, `SMTP_REPLY_TO`), reCAPTCHA (`RECAPTCHA_SECRET_KEY`, `NEXT_PUBLIC_RECAPTCHA_SITE_KEY`), Sentry (`SENTRY_DSN_BACKEND`, `SENTRY_API_TOKEN`, `SENTRY_WEBHOOK_SECRET`, `SENTRY_AUTH_TOKEN`, `NEXT_PUBLIC_SENTRY_DSN`), NELC LRS (`NELC_LRS_PASSWORD`, `NELC_LRS_PASSWORD_PROD`)
Non-sensitive config kept as secrets	`MAIL_FROM`, `MAIL_REPLY_TO`, `STORAGE_BUCKET_PUBLIC`, `STORAGE_BUCKET_PRIVATE`, `STORAGE_BUCKET_TRAINERS`

Deploy scripts feature-detect optional secrets with `gcloud secrets describe`, so a missing third-party key gracefully disables that integration rather than failing the deploy. Access is gated by the runtime service accounts listed in §9.

`infra/scripts/manage-secrets.sh` wraps `gcloud secrets` for listing, reading, writing and deleting secrets. Full per-secret rotation procedures live in `secrets-rotation-plan.md`.

11. Networking (VPC connector, private IPs, edge TLS)



Key details from source:

- Serverless VPC connector `physio-vpc-connector-me` is passed to `physioguidance-api`, `physioguidance-api-worker`, `physioguidance-cms` and every Cloud Run Job (`--vpc-connector=physio-vpc-connector-me --vpc-egress=all-traffic`).
- Cloud SQL is attached to Cloud Run via `--add-cloudsql-instances`; the Job scripts use the equivalent `--set-cloudsql-instances`.
- Edge TLS is hardened by `infra/scripts/setup-tls-hardening.sh`, which attaches SSL policy `physioguidance-ssl-policy` (`RESTRICTED` profile, min TLS 1.2) to `physioguidance-https-proxy` and points `physioguidance-web-http-proxy` at a redirect-only url-map (`physioguidance-web-redirect`) that 301s every HTTP hit to HTTPS.
- Frontend Cloud Run is `--allow-unauthenticated` (public). The api-worker and malware-scanner services are `--no-allow-unauthenticated` and have no public URL — reach them via the shared Redis queue, DB, or Eventarc respectively.

---

12. GitHub Actions to GCP (Workload Identity Federation)

Four deployment workflows live under `.github/workflows/`:

Workflow	Trigger	Runs
`deploy-backend.yml`	Push to `main` under `backends/**` or `infra/backend/**`, or manual	`bash infra/backend/deploy.sh --skip-pull`
`deploy-frontend.yml`	Push to `main` under `frontend/**` or `infra/frontend/**`, or manual	`bash infra/frontend/deploy.sh --skip-pull`
`deploy-worker.yml`	Push to `main` under `infra/worker/**`, manual, or on successful completion of `Deploy Backend`	`bash infra/worker/deploy.sh`
`deploy-strapi.yml`	Push to `main` under `strapi/**` or `infra/strapi/**`, or manual	`bash infra/strapi/deploy.sh --skip-pull`

Every workflow authenticates to GCP with `google-github-actions/auth@v2` using Workload Identity Federation — no long-lived JSON key on the runner. Three repo secrets drive it:

- `WIF_PROVIDER` — full resource name of the workload identity pool provider.
- `WIF_SERVICE_ACCOUNT` — the SA the workflow impersonates for the deploy.
- `GCP_PROJECT_ID`, `GCP_REGION` — passed through as env vars into `deploy.sh`.

Each workflow declares `permissions: id-token: write` so the GitHub OIDC token can be minted; the auth action exchanges it for a short-lived GCP access token. Concurrency is scoped per workflow so a second push cancels or queues cleanly.

---

13. Cost profile

Monthly cost breakdown lives in `cost-tally.md`. In summary, at current traffic the Cloud SQL instance is the largest line item, followed by Redis and Cloud Run. Cloud Tasks, Cloud Scheduler, Eventarc and Secret Manager are effectively free at this volume. Cloud Storage cost is dominated by egress; scan/scheduler traffic is negligible.

A budget alert on the project should be armed by the client's finance owner (see `budget-alert-status.md`).

---

14. Component status matrix

Using Amna's four-category vocabulary.

Component	Status	Notes
5 Cloud Run services deployed	Implemented	Verified against `infra/{web,api,worker,cms,malware-scanner}/deploy.sh`
3 Cloud Run Jobs (video-transcode, captions-generate, wcag-check)	Implemented	Provisioned by the three `infra/scripts/setup-*-job.sh` scripts
8 Cloud Tasks queues + OIDC dispatch	Implemented	`infra/scripts/setup-cloud-tasks.sh`
3 Cloud Scheduler jobs (reconciliation + BNPL poll)	Implemented	`infra/scripts/setup-reconciliation-cron.sh`
3 Eventarc triggers to malware scanner	Implemented	`infra/malware-scanner/deploy.sh`
Per-service runtime SAs on every Cloud Run service	Implemented	Verified in each `deploy.sh`
Cloud SQL private-socket access via `--add-cloudsql-instances`	Implemented	Verified in every backend/worker/job script
GCS quarantine bucket with uniform IAM + PAP	Implemented	`infra/malware-scanner/deploy.sh`
GCS bank-transfer receipts 10-year retention	Implemented	`infra/scripts/setup-bank-receipt-retention.sh`
GCS backups bucket 30-day lifecycle	Implemented	`infra/scripts/setup-backups.sh`
Edge TLS hardening (min TLS 1.2, HTTP to HTTPS redirect)	Implemented	`infra/scripts/setup-tls-hardening.sh`
Cloud Logging alerts (5xx, failed login, suspicious 403, malware)	Implemented	`infra/monitoring/*.json`
Workload Identity Federation for GitHub Actions	Implemented	Every workflow uses `google-github-actions/auth@v2` with WIF
Cloud Armor / WAF	Not Applicable	Edge SSL policy plus app-layer rate limiting covers the current threat model
Budget alert activation	Requires Client/Legal Action	Finance owner must arm the budget in the console; see `budget-alert-status.md`

15. Rebuild-from-zero procedure

Every step below has a script in this repo. Recovering the project from scratch is achievable from a clean laptop with `gcloud` and this git checkout.

1. Create the GCP project and link a billing account (one-time, manual).
2. Enable the APIs used across the deploys:

```
gcloud services enable run.googleapis.com sqladmin.googleapis.com \
  redis.googleapis.com eventarc.googleapis.com cloudtasks.googleapis.com \
  cloudscheduler.googleapis.com secretmanager.googleapis.com \
  storage.googleapis.com monitoring.googleapis.com \
  artifactregistry.googleapis.com cloudbuild.googleapis.com
```

3. Run `bash infra/scripts/setup-gcp.sh` to create the Artifact Registry repo and configure Docker.
4. Create the runtime service accounts listed in §9 (`web-runtime@` , `api-runtime@` , `worker-runtime@` , `strapi-runtime@` , `malware-runtime@` , `tasks-dispatcher@` , `reconciliation-cron@`) and grant each the roles indicated in the same section.
5. Create the Serverless VPC connector `physio-vpc-connector-me` in `me-central2` .
6. Run `bash infra/scripts/setup-database.sh` to create `physioguidance-db-me` and store `DATABASE_URL` in Secret Manager, then restore from the latest `physioguidance-backups` dump.
7. Create the Memorystore Redis instance in the VPC, populate `REDIS_HOST` and `REDIS_CA_CERT` in Secret Manager.
8. Run `bash infra/scripts/setup-storage.sh` , `bash infra/scripts/setup-backups.sh` , and `bash infra/scripts/setup-bank-receipt-retention.sh` to create and lifecycle-configure the six buckets.
9. Restore Secret Manager entries from an encrypted offline backup (the platform owner maintains this export).
10. Deploy the runtime in order: `infra/backend/deploy.sh` , `infra/worker/deploy.sh` , `infra/strapi/deploy.sh` , `infra/frontend/deploy.sh` , `infra/malware-scanner/deploy.sh` . Then `infra/scripts/setup-video-transcode-job.sh` , `infra/scripts/setup-captions-generate-job.sh` , `infra/scripts/setup-wcag-check-job.sh` for the three jobs.
11. Wire scheduling: `infra/scripts/setup-cloud-tasks.sh` and `infra/scripts/setup-reconciliation-cron.sh` .
12. Harden the edge: `infra/scripts/setup-tls-hardening.sh` ; re-apply the monitoring alert policies under `infra/monitoring/` per the README there.
13. Point the custom domains at Cloud Run via domain mappings.
14. Run the post-deploy verification checklist (`production-verification-checklist.md`).

Recovery from a clean project is on the order of a few hours for a familiar engineer; the backup restore step is the longest.

16. Where to look next

- How code moves from GitHub to production — `cicd-deployment.md`
- Environment variables and secrets per service — `environment-configuration-CONFIDENTIAL.md`
- Security controls in depth — `security-architecture.md`
- Backup and DR playbook — `regional-failure-playbook.md`
- Day-to-day operations — `platform-owner-guide.md`